**JSPM's**

**RAJARSHI SHAHU COLLEGE OF ENGINEERING**

**TATHAWADE, PUNE-33**

**DEPARTMENT OF COMPUTER ENGINEERING**

# MINI PROJECT REPORT

**ON**

# "IPO LISTING-GAIN PREDICTION USING MACHINE LEARNING"

**Submitted by**

Name: _____

Roll No.: _____

Class and Division: TY Div _____

Course: Machine Learning

Academic Year 2026–27 (Semester-I)

Submitted to: _____

# 1. Title

IPO Listing-Gain Prediction Using Machine Learning

# 2. Abstract

IPO listing-gain prediction is a machine-learning problem in which the listing performance of a company's Initial Public Offering (IPO) is estimated from its details before it lists on the stock exchange. This mini project demonstrates a complete workflow using Python, Pandas, Scikit-learn, and Streamlit. The input features include sector, issue size, price band, subscription of investor categories, grey-market premium, and company financial indicators. Missing values are filled using the Expectation-Maximization (EM) algorithm, a Hidden Markov Model identifies the market mood, and K-Means and K-Medoids group the IPOs. Linear, Polynomial, and Logistic Regression, SVM, k-NN, and a boosting model are then trained and compared using MSE, $R^2$, accuracy, precision, recall, and AUC. A Streamlit interface lets the user enter IPO details and view the expected listing gain and verdict. This project is for educational purposes only and is not investment advice.

# 3. Introduction

When an IPO opens, investors must decide whether to apply. The listing price may be higher or lower than the issue price, and many investors decide using rumours or the unofficial grey-market premium. Machine learning can learn from past IPOs and their actual listing results and use these patterns to estimate the likely outcome of a new IPO. This project presents a small educational implementation for the Indian IPO market.

The project uses algorithms studied in the Machine Learning Laboratory. Regression models estimate the listing gain percentage, while classification models predict whether the listing will be positive and whether the IPO should be marked Avoid, Neutral, or Apply. The EM algorithm handles missing data, the Hidden Markov Model estimates the market mood (Cold, Warm, or Hot), and clustering groups similar IPOs. The data is divided by listing date, so older IPOs are used for training and recent IPOs for testing.

# 4. Problem Statement

To develop a machine-learning application that predicts the listing gain of an Indian IPO based on its sector, issue size, subscription, grey-market premium, and financial details. The system should train and evaluate regression and classification models and provide a user-friendly interface for entering IPO details and displaying the predicted gain, the verdict, and similar past IPOs.

# 5. Objectives

- To understand IPO listing prediction as a supervised regression and classification task.
- To prepare a structured dataset of past IPOs and handle missing values using the EM algorithm.
- To estimate the IPO market mood using a Hidden Markov Model (Forward and Viterbi algorithms).
- To group IPOs using K-Means and K-Medoids clustering.
- To train Linear, Polynomial, Logistic Regression, SVM, k-NN, and a boosting model.
- To evaluate model performance using MSE, $R^2$, accuracy, precision, recall, and AUC.

- To build an interactive prediction interface using Streamlit.

# 6. Data Set and Libraries Required

## 6.1 Dataset

The dataset contains 35 past Indian IPOs collected from public sources such as NSE, BSE, and Chittorgarh. Each record has the IPO details and the listing result.

| Column | Description | Type |
|---|---|---|
| sector | Industry sector of the company | Categorical |
| issue_size_cr | Total issue size in ₹ crore | Numerical |
| price_band_upper | Upper price of the price band (₹) | Numerical |
| qib_sub, nii_sub, retail_sub | Subscription multiple of each investor category | Numerical |
| gmp_last_day | Grey-market premium on the last day | Numerical |
| revenue_growth_pct, profit_growth_pct | Company growth indicators | Numerical |
| pe_vs_peers | Issue P/E compared with sector peers | Numerical |
| listing_gain_pct | Target: listing gain in percent | Numerical |
| positive_listing | Target: 1 if gain is positive, else 0 | Binary |
| verdict | Target: Avoid / Neutral / Apply | Categorical |

## 6.2 Libraries and Tools

- Python 3.11 — programming language.
- Pandas and NumPy — data handling and numerical computation.
- Scikit-learn — preprocessing, models, and evaluation.
- Matplotlib — plots.
- Joblib — saving and loading trained models.
- Streamlit — creating the web interface.
- Visual Studio Code or Jupyter Notebook — development environment.

# 7. Approach / Methodology

1. **Data loading:** Read the IPO data with Pandas, check the required columns, and create the target columns.
2. **Train/test split:** Sort the IPOs by listing date and use the oldest 75% for training and the newest 25% for testing.
3. **Missing values:** Fill missing values using the EM algorithm, fitted on the training data.
4. **Preprocessing:** One-hot encode the sector and scale the numerical features.
5. **Market mood:** Use a Hidden Markov Model with the Forward and Viterbi algorithms to find the Cold, Warm, or Hot state of the IPO market.

6. **Clustering:** Group the IPOs using K-Means and K-Medoids and add the cluster label as a feature.
7. **Model training:** Train Linear, Polynomial, and Logistic Regression, SVM, k-NN, and AdaBoost / Gradient Boosting.
8. **Evaluation:** Calculate MSE and $R^2$ for regression and accuracy, precision, recall, and AUC for classification on the test set.
9. **Application:** Save the models with Joblib and use Streamlit to take IPO details and show the predictions.

# 8. Implementation

The implementation is provided as a project folder on GitHub. The main folders are data, notebooks, src, app, and models. The following commands install the dependencies and launch the Streamlit application.

GitHub repository: https://github.com/pranavkale1108/IPO-ListingGain-Predictor-

## 8.1 Installation

```
python -m venv .venv
Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
```

## 8.2 Train the models

Run the notebooks in order. They clean the data, train all models, print the evaluation metrics, and save the models in the models folder.

## 8.3 Run the Streamlit application

```
python -m streamlit run app/streamlit_app.py
```

Open the local URL printed in the terminal, usually http://localhost:8501. Enter the IPO details and click predict. The app shows the expected listing gain, the verdict, similar past IPOs, and the market mood. The application is also deployed on Streamlit Community Cloud: http://localhost:8501

## 8.4 Core code

```
def time_split(df, train_frac=0.75):
    df = df.sort_values("listing_date").reset_index(drop=True)
    cut = int(len(df) * train_frac)
    return df.iloc[:cut], df.iloc[cut:]

train, test = time_split(df)
model = GradientBoostingClassifier(random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

## 9. Results

The models are trained and evaluated on the test set (the most recent IPOs).

| Model | Task | Metric | Value from your run |
|---|---|---|---|
| Linear Regression | Gain % | $R^2$ | 0.4611 |
| Polynomial Regression | Gain % | $R^2$ | 0.6283 |
| Logistic Regression | Positive / negative | Accuracy | 55.56% |
| SVM | Avoid / Neutral / Apply | Accuracy | 44.44% |
| k-NN (best K) | Positive / negative | Accuracy | 55.56% |
| AdaBoost / Gradient Boosting | Positive / negative | Accuracy | 44.44% |

Expected functional result: the app accepts IPO details and displays the expected listing gain, the verdict, similar past IPOs, and the market mood. Because the dataset is small, the results show the working of the implementation and should not be treated as investment advice.

Suggested screenshots to add after running the project: (1) terminal or notebook showing the evaluation metrics; (2) Streamlit form with sample IPO details; and (3) the prediction result displayed by the app.

## 10. Conclusion

This mini project demonstrates an end-to-end machine-learning workflow for predicting the listing performance of IPOs. It uses Pandas for data handling, Scikit-learn for preprocessing, models, and evaluation, EM, HMM, and clustering for feature creation, Joblib for saving models, and Streamlit for the user interface. The main limitation is the small dataset and the noisy grey-market premium. Future work should use more IPO data, include more features, compare more algorithms, and validate predictions on later IPOs before using the application for any real decision.

## References


- Scikit-learn documentation: https://scikit-learn.org/
- Streamlit documentation: https://docs.streamlit.io/
- Pandas documentation: https://pandas.pydata.org/docs/
- NSE India: https://www.nseindia.com/
- BSE India: https://www.bseindia.com/
- Chittorgarh: https://www.chittorgarh.com/